

TWO DAYS SUMMER SCHOOL ON

SUPERCOMPUTING FOR MODELING AND SIMULATION



Lecture 1 · Section 2

Fundamentals of Modeling & Simulation

From real-world systems to computational models — and
why large simulations need HPC

Speaker: Amna Haider

Co-Founder Pakistan Supercomputing

C.E.O. UCERD Pvt Ltd

22-23 AUGUST 2026 · NAMAL UNIVERSITY, MIANWALI

What You Should Understand After This Section

A practical foundation before we move into HPC and scientific software.

Model

Explain what a model represents and why simplification is necessary.

Simulation

Differentiate a model from the act of simulating it.

Four paradigms

Distinguish mesh-based, particle-based, agent/event-based, and hybrid/multiscale modeling.

Connect to HPC

Explain why realistic models quickly outgrow a laptop or workstation.

Why Do We Build Models?

Because experiments in the real world are not always practical, affordable, safe or repeatable.

UNDERSTAND

Explore cause and effect

PREDICT

Estimate future behaviour

TEST

Try many scenarios safely

OPTIMIZE

Search for better designs

REDUCE COST

Avoid excessive prototypes

REDUCE RISK

Study dangerous/extreme cases

**100 physical
prototypes**

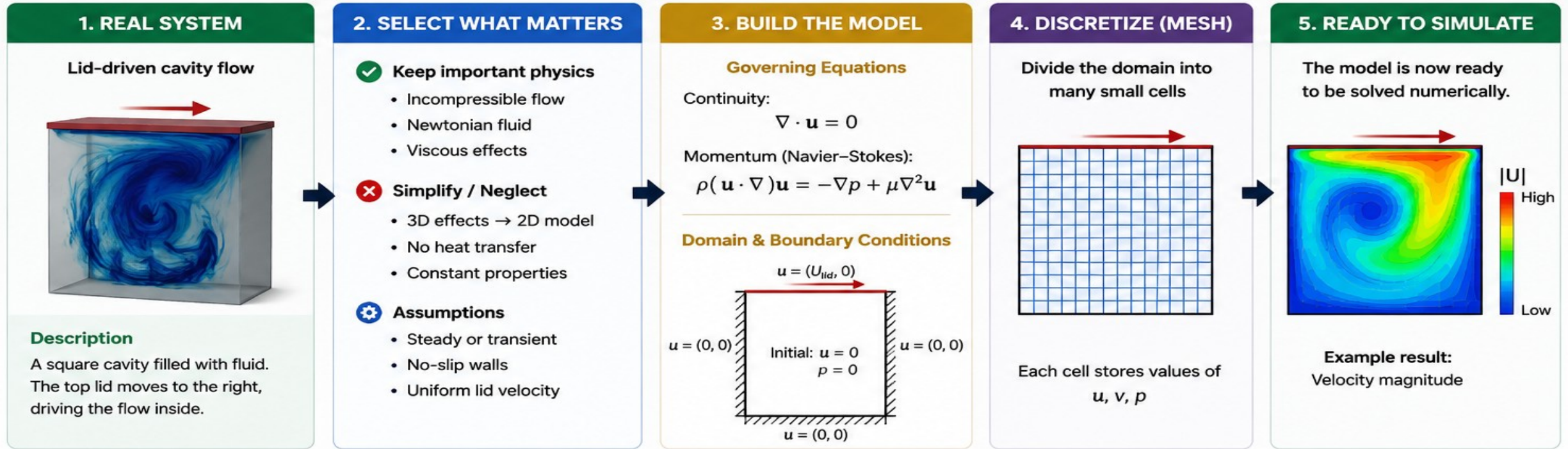
vs

**100 virtual design
experiments**

→ faster iteration, earlier failure detection, lower development cost

Example of Modeling

From a real system to a computational model



What is the Model?



A simplified mathematical and computational representation of the lid-driven cavity flow that captures the essential behavior needed to answer our question.

Model elements

- Equations
- Geometry
- Boundary conditions
- Assumptions
- Discretization (mesh)

Key Takeaway



Modeling describes the system: deciding what to include, what to simplify, and how to represent it.

Simulation runs the model to obtain results.



In simple words:

We take a complex real system, keep what matters, make simplifying assumptions, write equations, create a model, discretize it, and then solve it on a computer.



Simplify



Model



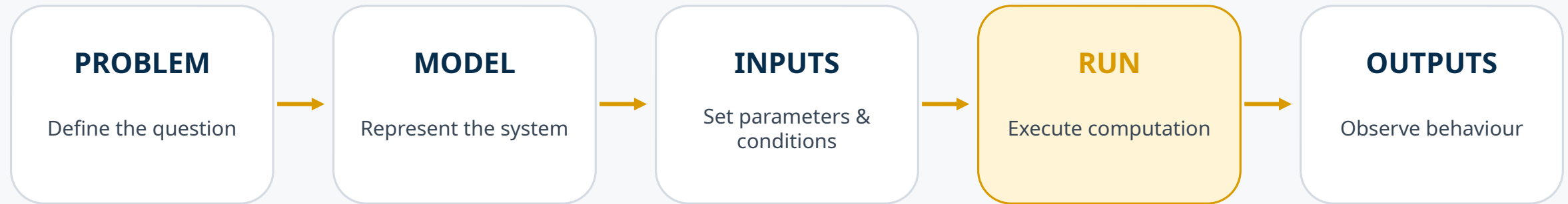
Simulate



Insight

What Is Simulation?

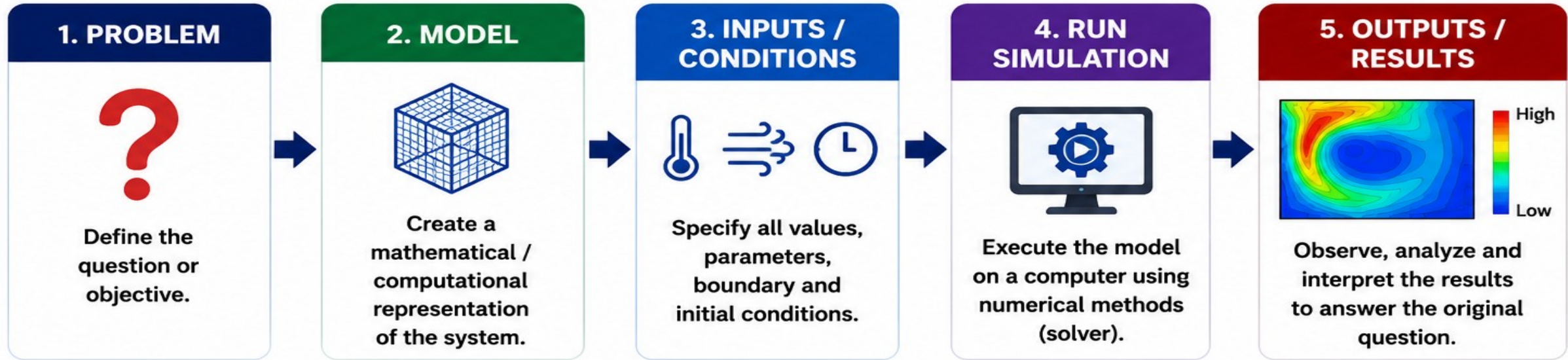
Simulation is the execution of a model to observe how the modeled system behaves.



WHAT IS SIMULATION?



Simulation is the **execution of a model** under selected **inputs and conditions** to **observe how the modeled system behaves**.



In one line:

Simulation is a computational experiment performed on a model.

Modeling vs Simulation

Related concepts — but not the same thing.

MODELING

- Defines what the system is
- Selects variables and relationships
- Introduces equations, rules, geometry and assumptions
- Answers: “How should we represent the system?”

SIMULATION

- Executes the model
- Applies inputs, boundary conditions and time steps
- Calculates the system response
- Answers: “What happens under these conditions?”

Modeling describes. Simulation experiments.

Physical Experiment vs Computational Simulation

Both are valuable; simulation helps us explore more cases before building or testing physically.

PHYSICAL EXPERIMENT

1 Real system
Existing or built hardware

2 Prototype / setup
Material + instrumentation

3 Experiment
Apply load / flow / condition

4 Measurement
Sensors & observations

COMPUTATIONAL SIMULATION

1 Real problem
What do we need to know?

2 Mathematical model
Equations + assumptions

3 Computer simulation
CPU / GPU / cluster

4 Numerical results
Fields, trajectories, statistics

Simulation is especially useful when the real experiment is expensive, slow, dangerous, tiny, huge, or difficult to repeat.

How Do We Classify Models and Simulations?

Use two layers: characteristics of behaviour, then the computational paradigm used to represent the system.

B. COMPUTATIONAL PARADIGM

- 1 Mesh-Based / Continuum
- 2 Particle-Based / Atomistic
- 3 Agent-Based / Discrete-Event
- 4 Hybrid / Multiscale

Important: these classifications can overlap. A mesh-based CFD model, for example, may also be dynamic, continuous and transient.

Four Major Computational Modeling & Simulation Paradigms

A simple four-category view for this summer school.

1

MESH-BASED / CONTINUUM

Divide space into cells or elements

CFD · FEM · heat transfer

2

PARTICLE-BASED / ATOMISTIC

Track particles, atoms or bodies

MD · DEM · SPH · N-body

3

AGENT / DISCRETE-EVENT

Track entities, rules and events

Traffic · queues · manufacturing

4

HYBRID / MULTISCALE

Combine methods or scales

DFT + MD + continuum + system

The first question is usually: What are we representing — space, particles, agents/events, or several scales together?

TYPES OF SIMULATION

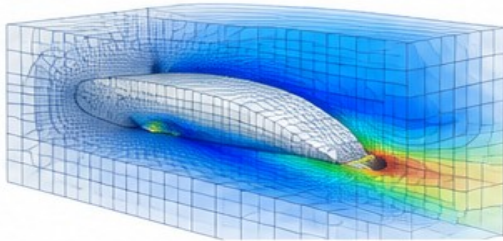
Depending on Computational Modeling Method



The way we **represent the system** in a computer determines the type of simulation we use.

1 MESH-BASED / CONTINUUM SIMULATION

The physical space is divided into small cells or elements. Governing equations are solved over the mesh.



Examples:

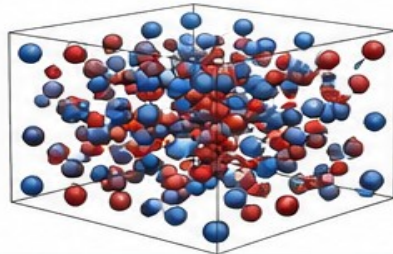
- CFD (fluid flow)
- FEM (structural/solid mechanics)
- Heat transfer
- Electromagnetics
- Weather / climate modeling

Software:

OpenFOAM, ANSYS, COMSOL, Abaqus, FLUENT

2 PARTICLE-BASED / ATOMISTIC SIMULATION

The system is represented by a large number of particles (atoms, molecules or bodies). Interactions between particles are computed.



Examples:

- Molecular Dynamics (MD)
- Discrete Element Method (DEM)
- Smoothed Particle Hydrodynamics (SPH)
- N-body simulations (astrophysics)

Software:

GROMACS, LAMMPS, HOOMD-blue, LAMMPS, DualSPHysics

3 AGENT-BASED / DISCRETE-EVENT SIMULATION

The system is represented by individual entities (agents) and their rules, interactions or events over time.



Examples:

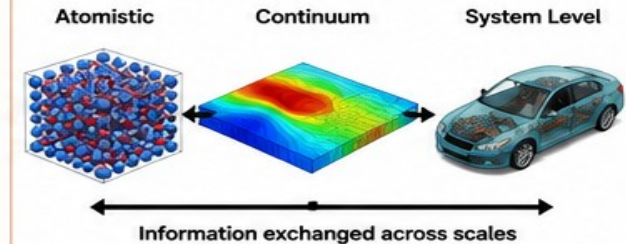
- Traffic and transportation
- Manufacturing and supply chains
- Epidemics and social behavior
- Communication networks
- Queueing and service systems

Software:

AnyLogic, Arena, SimPy, NetLogo, OMNeT++, NS-3

4 HYBRID / MULTISCALE SIMULATION

Two or more modeling approaches or physical scales are combined in one simulation.



Examples:

- DFT + MD
- Particle + Continuum (CFD / FEM)
- FEM + CFD fluid-structure interaction
- Digital twin / system of systems

Software:

LAMMPS + OpenFOAM, ANSYS Multiphysics, Co-Simulation tools



Key Takeaway

The choice of computational modeling method depends on:



What we want to study



Level of detail needed



Available computational resources



Time and accuracy requirements



Remember

A simulation can belong to several categories simultaneously.

Example:

Transient CFD = Deterministic + Dynamic + Continuous + Transient + Mesh-Based

MESH-BASED SIMULATION OF A CAR (CFD EXAMPLE)

Mesh-based (continuum) simulation solves the governing equations on a discretized domain (mesh) to predict flow behavior around the car.

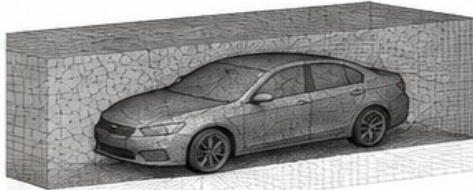
1. GEOMETRY

3D CAD model of the car



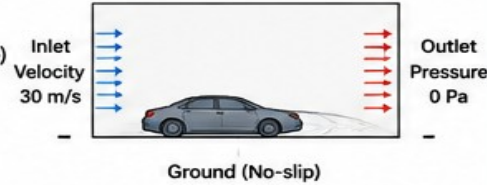
2. MESH (DISCRETIZATION)

Computational domain divided into millions of small cells



3. PHYSICS & BOUNDARY CONDITIONS

- Governing Equations: Navier-Stokes
- Turbulence Model: $k-\omega$ SST
- Inlet: Velocity = 30 m/s
- Outlet: Pressure = 0 Pa (gauge)
- Ground: No-slip wall
- Car Surface: No-slip wall
- Air: Density = 1.225 kg/m^3 , Viscosity = $1.789\text{e-}5 \text{ Pa}\cdot\text{s}$



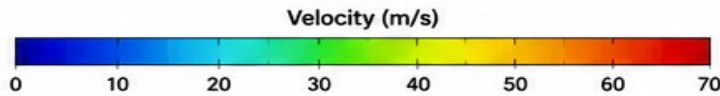
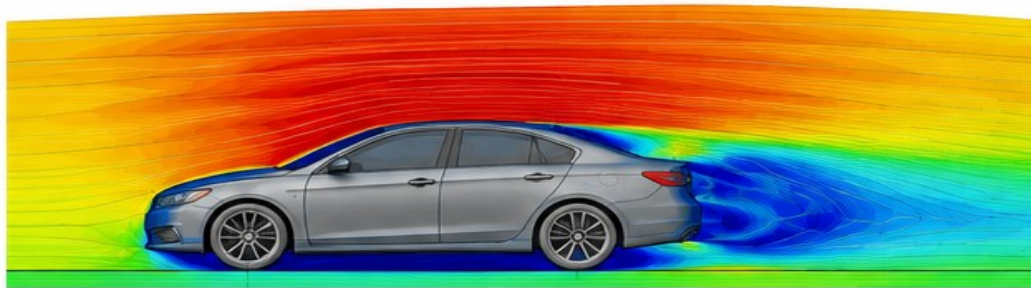
4. SOLVER (COMPUTATION)

Equations are solved iteratively on the mesh using CFD solver

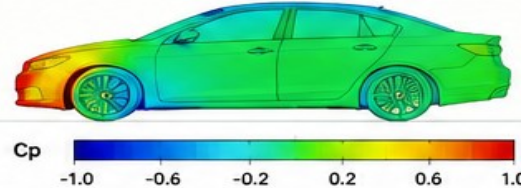
OpenFOAM®

5. RESULTS (FLOW AROUND THE CAR)

VELOCITY CONTOUR (m/s)



PRESSURE COEFFICIENT (C_p)



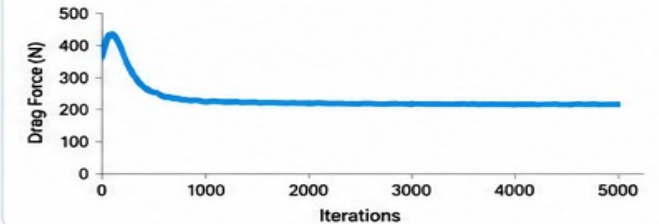
STREAMLINES



TURBULENT KINETIC ENERGY (m^2/s^2)



DRAG FORCE vs ITERATIONS



KEY OUTPUTS



Drag Force
198.7 N



Lift Force
35.2 N



Drag Coefficient (C_d)
0.294



Lift Coefficient (C_l)
0.052

APPLICATIONS

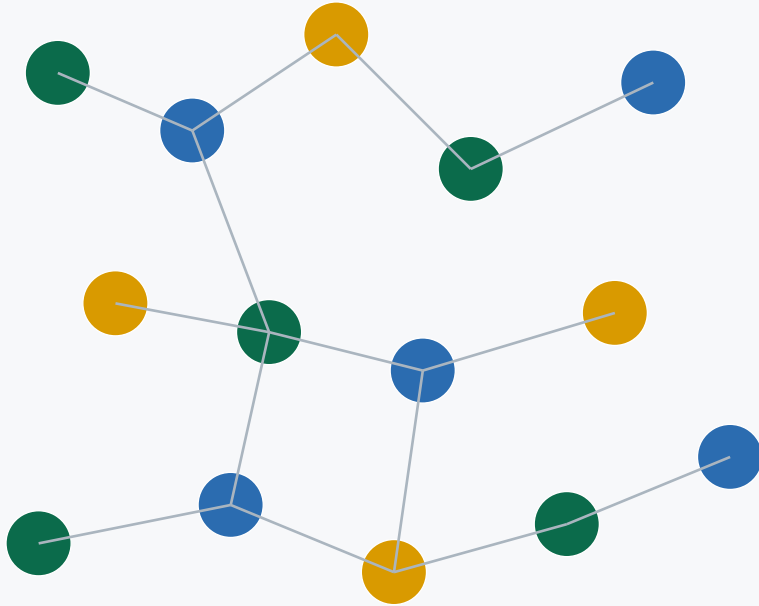
- ✓ Improve fuel efficiency (reduce drag)
- ✓ Enhance vehicle stability and handling
- ✓ Optimize shape for better aerodynamic performance
- ✓ Support design decisions before physical testing



Mesh-based CFD simulation helps engineers predict and optimize airflow around a car with high accuracy before building prototypes.

Type 2 — Particle-Based / Atomistic Modeling

Represent the system using individual particles, atoms, molecules or bodies.



PARTICLES → INTERACTIONS / FORCES → MOTION → NEW POSITIONS

COMMON METHODS

- Molecular Dynamics (MD)
- Discrete Element Method (DEM)
- Smoothed Particle Hydrodynamics (SPH)
- N-body simulation

Applications: proteins, materials, granular systems, fluids, astrophysics

Software examples: GROMACS, LAMMPS

Simple message: instead of dividing space, track the objects or particles themselves.

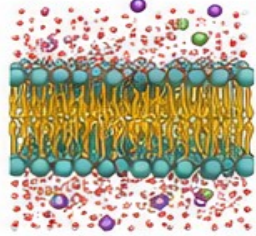
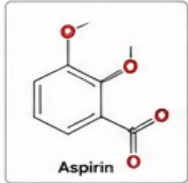
PARTICLE-BASED SIMULATION: ASPIRIN ABSORPTION MODEL IN GROMACS

Molecular Dynamics (MD) simulation of aspirin transport across a lipid bilayer (intestinal membrane)

1. SYSTEM DEFINITION

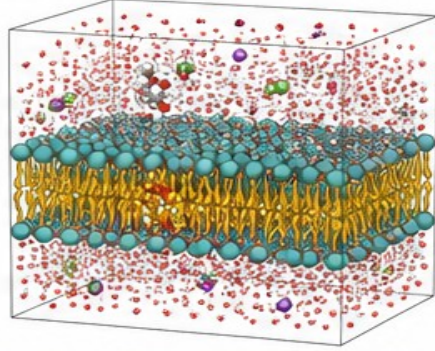
Build the molecular system

- Aspirin (acetylsalicylic acid)
- Lipid bilayer (POPC)
- Water (TIP3P)
- Ions (0.15 M NaCl)



2. INITIAL CONFIGURATION

Solvate and add ions

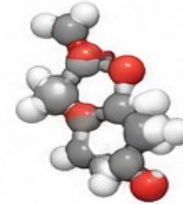


3. FORCE FIELD & PARAMETERS

Force field: CHARMM36

Aspirin parameters:
CGenFF/GAFF

- Carbon
- Hydrogen
- Oxygen
- Phosphorus
- Nitrogen
- Sodium
- Chloride



4. SIMULATION IN GROMACS

MD simulation (NPT ensemble)

Typical settings:

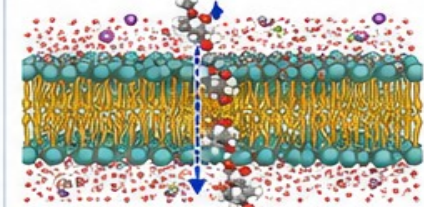
- Temperature: 310 K
- Pressure: 1 bar
- Time step: 2 fs
- Cutoff: 1.0 nm
- PME for long-range electrostatics
- Duration: 300 ns



5. OBSERVABLES

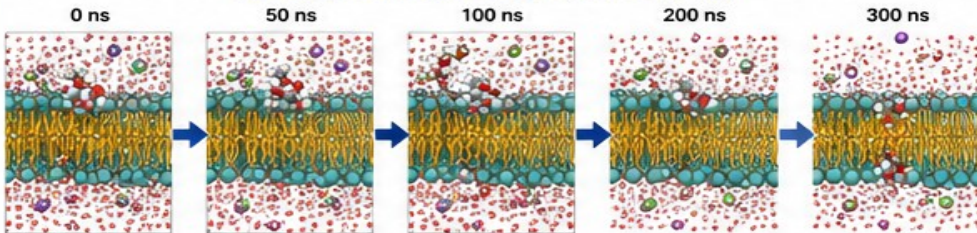
Track aspirin absorption across the membrane

- Position of aspirin
- Orientation
- Diffusion
- Permeation events



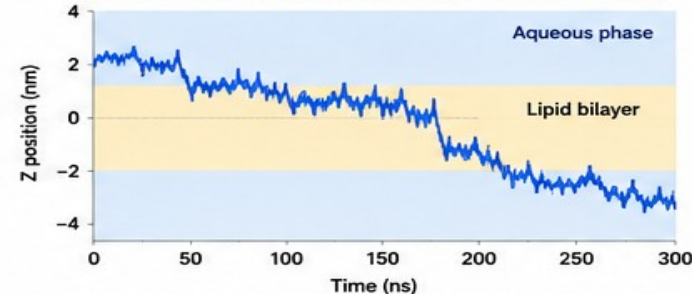
6. EXAMPLE SIMULATION RESULTS

A. TRAJECTORY SNAPSHOTS (Time evolution)

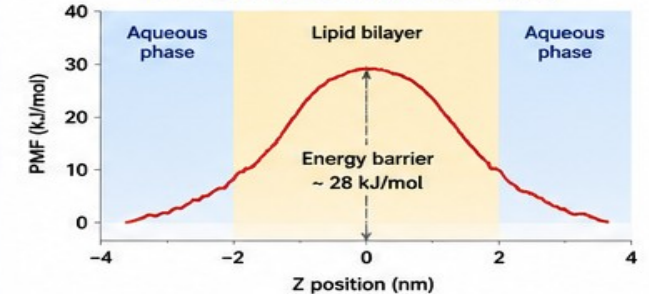


Aspirin diffuses from the aqueous phase, interacts with the bilayer, and permeates to the other side.

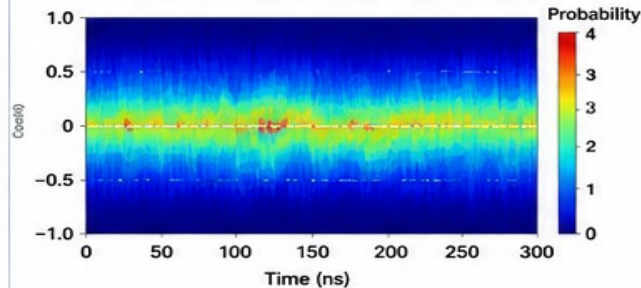
B. ASPIRIN POSITION ALONG MEMBRANE NORMAL (Z)



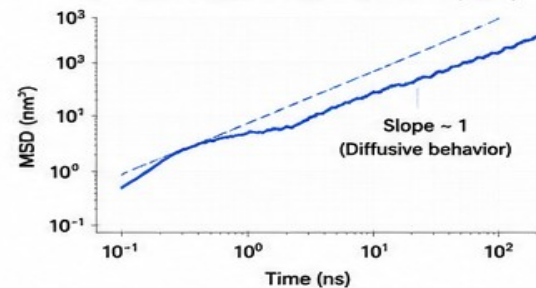
C. POTENTIAL OF MEAN FORCE (PMF)



D. ASPIRIN ORIENTATION DISTRIBUTION

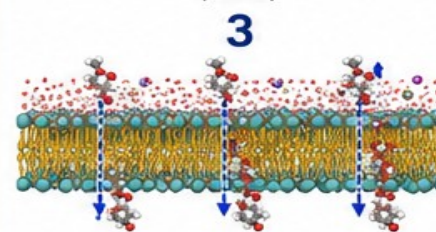


E. MEAN SQUARE DISPLACEMENT (MSD)



F. PERMEATION EVENTS

Number of successful permeation events (300 ns)



7. APPLICATION / INSIGHTS

- ✓ Aspirin can cross the lipid membrane via passive diffusion.
- ✓ Energy barrier (~28 kJ/mol) indicates moderate permeability.
- ✓ MD simulation helps understand absorption mechanism at molecular level.
- ✓ Useful for drug formulation and bioavailability prediction.

Particle-based MD captures molecular interactions and provides atomistic insights into drug absorption.



Software: GROMACS
Force Field: CHARMM36 + CGenFF



System Size: ~ 100,000 atoms
Simulation Time: 300 ns



Platform: Linux / HPC / GPU
Command example: `gmx mdrun -deffnm aspirin_absorption`

Mesh-Based vs Particle-Based — The Key Difference

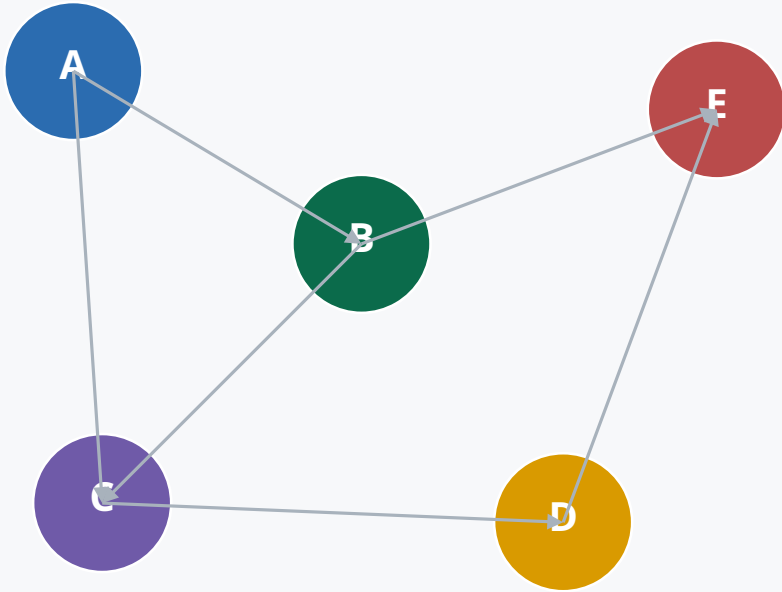
Two very different ways of representing a physical system.

COMPARISON	MESH-BASED	PARTICLE-BASED
What is divided?	Space / domain	Matter / particles
Basic unit	Cell / element / node	Particle / atom / molecule
Typical output	Fields: velocity, pressure, temperature	Trajectories, positions, energies
Typical methods	FDM · FEM · FVM	MD · DEM · SPH
Example	Airflow around a fan blade	Atoms inside a protein
Example software	OpenFOAM / ANSYS	GROMACS / LAMMPS

Mesh-based: divide SPACE. Particle-based: represent MATTER.

Type 3 — Agent-Based / Discrete-Event Modeling

Represent individual entities and the rules or events that change their state.



**ENTITIES / AGENTS + RULES + EVENTS → SYSTEM
BEHAVIOUR**

WHERE IT IS USED

Examples of agents/entities

- People, vehicles, machines, network packets, jobs

Applications

- Traffic and mobility
- Manufacturing and supply chains
- Epidemics and population models
- Communication networks
- HPC job scheduling

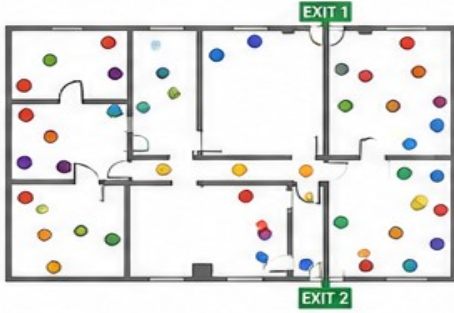
AGENT-BASED SIMULATION EXAMPLE IN SIMPY

Pedestrian Movement in a Building (Crowd Simulation)

We simulate individual pedestrians (agents) moving from different rooms to exit gates.

1. PROBLEM

People in different rooms want to reach exit gates.
They avoid congestion and choose shortest available paths.



2. AGENTS

Each pedestrian is an autonomous agent with its own behavior.



Attributes

- id
- origin
- destination (exit)
- position
- speed



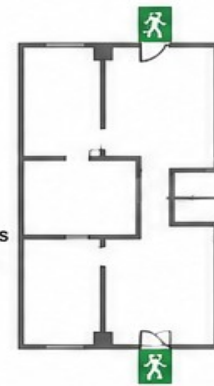
Behavior

- choose path
- move
- avoid crowd
- reach exit

3. ENVIRONMENT

The building layout is the simulation environment.

- Rooms
- Corridors
- Doorways
- Exit gates
- Capacity limits (people that can pass through a doorway per time unit)



4. MODEL LOGIC

Discrete-event simulation using SimPy processes.



Agent is created in origin room



Agent selects path to exit (shortest available)



Agent moves through corridors and doorways (seize resource)



Agent reaches exit and leaves system

5. IMPLEMENTATION (SIMPY)

Key SimPy concepts used



Agent = Process
(env.process(agent(env, ...)))



Doorway = Resource
(capacity = k people)



Movement = Timeout
(env.timeout(travel_time))



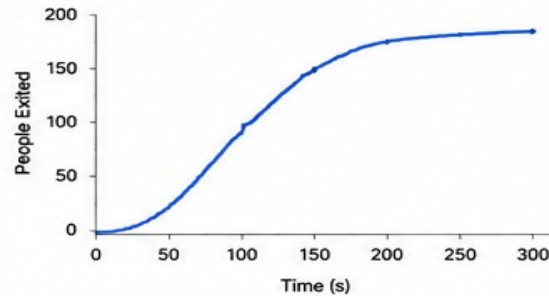
Statistics collection
(entry time, exit time, wait time)

6. SIMULATION RESULTS (EXAMPLE RUN)

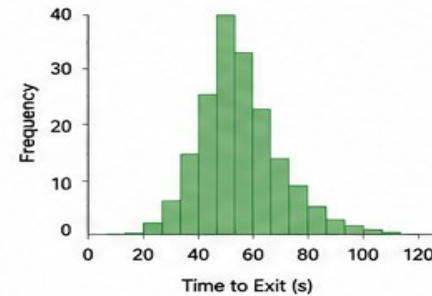
A. Snapshot of Simulation (Time = 280 s)



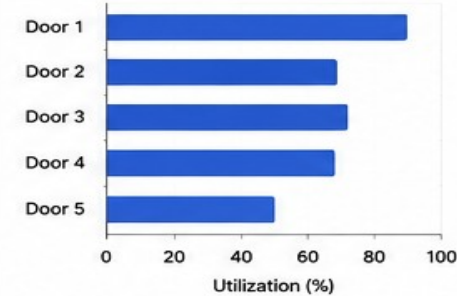
B. Number of People Who Exited Over Time



C. Average Time to Exit



D. Doorway Utilization



7. INSIGHTS

- ✓ Identify congested areas and critical doorways.
- ✓ Evaluate building designs and evacuation strategies.
- ✓ Test different scenarios (e.g., more exits, wider corridors).
- ✓ Improve safety and reduce evacuation time.



8. SIMPY CODE SNIPPET (Simplified)

```
import simpy, random
NUM_PEOPLE = 200
env = simpy.Environment()
door1 = simpy.Resource(env, capacity=3)
door2 = simpy.Resource(env, capacity=3)
```

```
def agent(env, name, origin, exit_door):
    yield env.timeout(random.expovariate(1/2)) # decide start time
    # move through corridor
    with exit_door.request() as req:           # request doorway
        yield req
    yield env.timeout(random.expovariate(1/1)) # pass through
    # exited
```

```
for i in range(NUM_PEOPLE):
    door = door1 if random.random() < 0.5 else door2
    env.process(agent(env, f'P{i}', 'RoomA', door))

env.run(until=300)
```

9. HOW TO RUN



```
$ python simulation.py
```

Output:

Total people exited: 184
Average time to exit: 57.3 s

Type 4 — Hybrid & Multiscale Modeling

Many real problems require more than one method or more than one physical scale.

ELECTRONIC

DFT / quantum

ATOMIC

Molecular dynamics

MATERIAL

Particle / continuum

COMPONENT

FEM / CFD

SYSTEM

System simulation / digital twin

WHY COMBINE SCALES?

A single method may not capture every important phenomenon.

Examples

- Battery: electrons → atoms → material → cell
- Drug: molecule → protein → cell → patient
- Aircraft: material → component → complete vehicle

Hybrid models trade simplicity for realism.

MULTISCALE = connect information across scales. HYBRID = combine different modeling approaches.

HYBRID SIMULATION: EFFECT OF AIR FLOW ON A CAR WITH ATOMIC MODEL OF CAR PROFILE

Coupling Continuum CFD (Air Flow) + Atomistic MD (Surface) → Multi-Scale Insights

1. CONCEPT

Air flows around the car (continuum scale).
Shear stress and pressure on the car surface
affect surface atoms (atomic scale).
Hybrid coupling transfers information
between the two scales.

CONTINUUM SCALE (CFD)



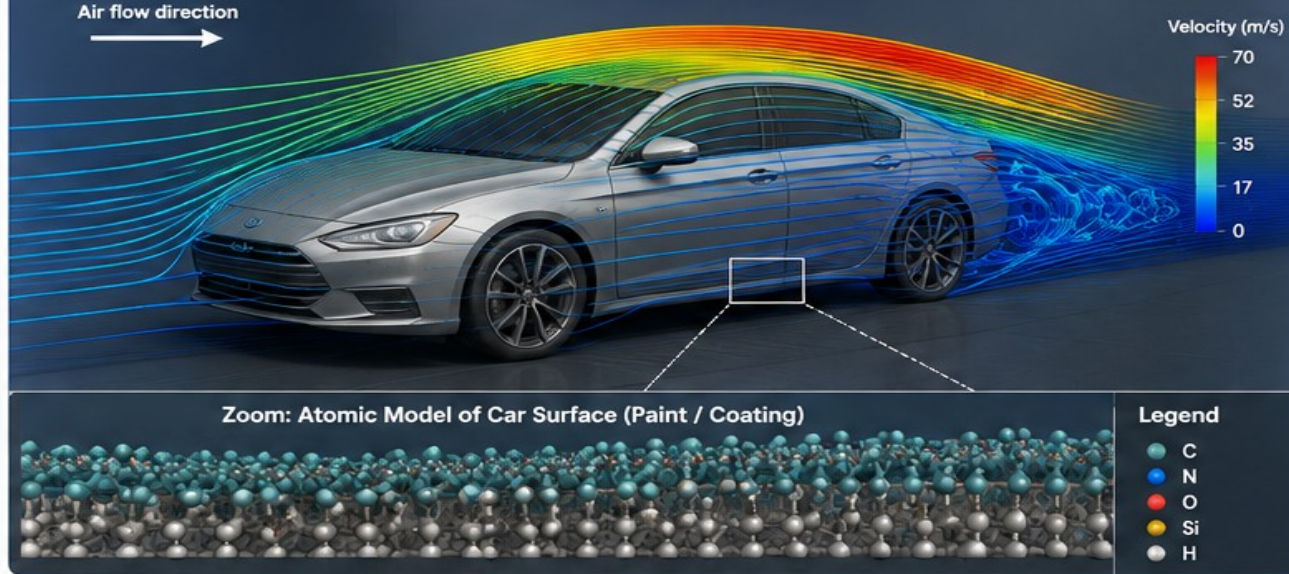
Pressure (P), Shear Stress (τ)
Heat Flux (q)

Surface Roughness (from MD)
Effective Slip Length

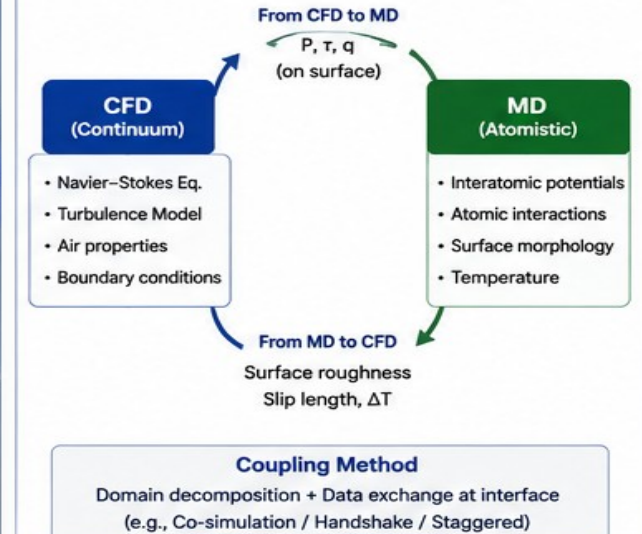
ATOMIC SCALE (MD)



2. HYBRID MODEL OVERVIEW

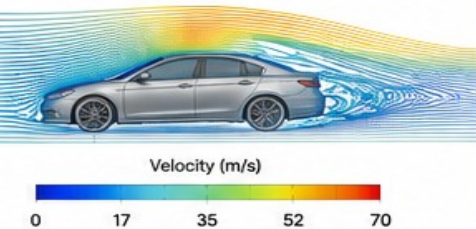


3. MULTI-SCALE COUPLING

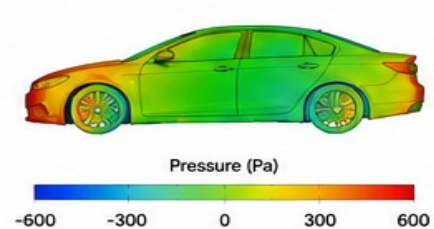


4. RESULTS

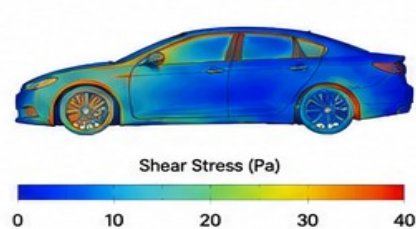
A. AIR VELOCITY AROUND CAR (CFD)



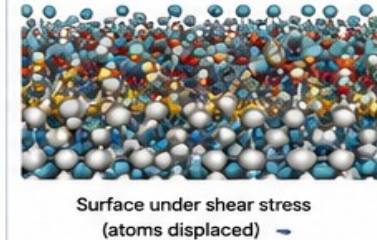
B. SURFACE PRESSURE DISTRIBUTION (CFD)



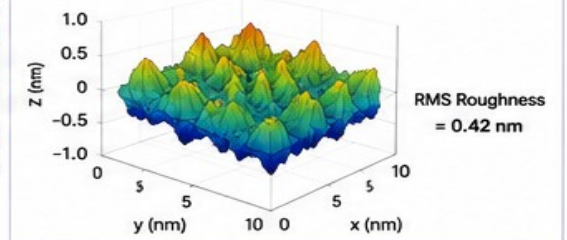
C. WALL SHEAR STRESS (CFD)



D. ATOMIC STRUCTURE (MD)



E. SURFACE ROUGHNESS (FROM MD)



5. PERFORMANCE IMPACT (EXAMPLE)

Parameter	CFD Only (Smooth Wall)	Hybrid (CFD + MD) (Realistic Surface)	Difference
Drag Coefficient (Cd)	0.285	0.297	+4.2%
Lift Coefficient (Cl)	0.012	0.015	+25.0%
Avg Wall Shear Stress (Pa)	18.7	20.3	+8.6%
Separation Length (m)	1.23	1.07	-13.0%



Insight

Atomic-scale surface
roughness increases drag
and changes flow
separation.

6. APPLICATIONS



- Accurate prediction of aerodynamic drag
- Design of low-drag car shapes and surfaces



- Study of surface coatings, roughness, and contamination

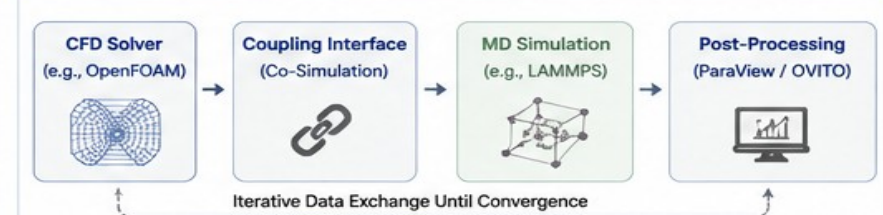


- Improved fuel efficiency and EV range estimation



- Virtual testing before physical prototyping

7. TOOLS & WORKFLOW



Why Does Simulation Become Computationally Expensive?

Realism usually increases the number of calculations in several dimensions at once.

MORE ELEMENTS / PARTICLES

Higher spatial or molecular resolution

SMALLER TIME STEP

More updates per simulated second

MORE PHYSICS

Coupled equations / complex interactions

MORE SCENARIOS

Design sweeps, uncertainty, optimization

LARGER DOMAIN

More space / molecules / components

LONGER SIMULATION

More time steps / trajectories

MORE REALISM → MORE COMPUTATION → NEED FOR PARALLELISM

Problem Size → Computational Requirement

A simple intuition: doubling model resolution can increase work far more than 2×.

Laptop

Small model

Workstation

Medium model

GPU server

Large model

HPC cluster

Very large model

Supercomputer

National / extreme-scale model

The correct platform depends on model size, solver efficiency, memory, runtime target and parallel scalability.

From Simulation to HPC

Why large scientific simulations need parallel computers



Simulation becomes an HPC problem when:

- Millions of cells / particles
 - Many equations & iterations
 - High memory requirement
 - Long runtime on one CPU
- ⇒ **Divide the work and solve it simultaneously**

KEY IDEA

Continuous physics → numerical model → large computational workload → parallel execution on HPC

Navier–Stokes Equations for Airflow Around a Car

The Navier–Stokes equations describe the motion of a fluid (air) around the car.



Navier–Stokes Equations for Incompressible Airflow

Continuity Equation

$$\nabla \cdot \mathbf{u} = 0$$

Momentum Equation

$$\rho \left(\frac{\partial \mathbf{u}}{\partial t} + \mathbf{u} \cdot \nabla \mathbf{u} \right) = -\nabla p + \mu \nabla^2 \mathbf{u}$$

- $\mathbf{u}$ = velocity vector of air (m/s)
- ρ = air density (kg/m³)
- p = pressure (Pa)
- μ = dynamic viscosity of air (Pa·s)
- ∇ = gradient operator
- ∇^2 = Laplacian operator

In words:

The continuity equation ensures mass conservation.

The momentum equation balances the change in momentum of air with pressure forces and viscous (friction) forces.

Purpose

Solve these equations numerically on a discretized mesh to predict airflow patterns, pressure distribution, drag and lift on the car.

NAVIER-STOKES → PARALLEL COMPUTING

Discretize the fluid domain into cells, then solve the same local equation on many MPI ranks.

1 CONTINUOUS PHYSICS

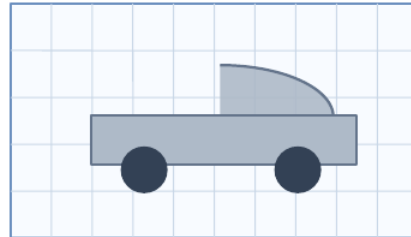
Incompressible flow

$$\nabla \cdot \mathbf{u} = 0$$
$$\rho(\partial \mathbf{u} / \partial t + (\mathbf{u} \cdot \nabla) \mathbf{u}) = -\nabla p + \mu \nabla^2 \mathbf{u}$$

One coupled PDE system over the whole air domain

2 DISCRETIZE THE DOMAIN

Air box around the car



Millions of finite-volume cells

3 LOCAL CELL EQUATION

Finite-volume form

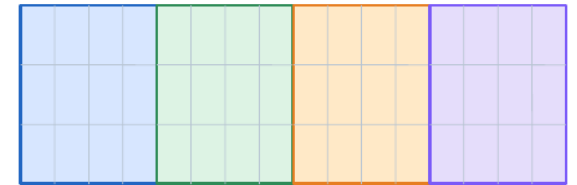


$$a_p \varphi_p = \sum a_n \varphi_n + b_p$$

Each cell depends mainly on neighboring cells

4 DECOMPOSE ACROSS MPI

Same mesh, split into subdomains



Rank 0 Rank 1 Rank 2 Rank 3

Each rank owns cells; interfaces exchange neighbor data

AT EACH ITERATION / TIME STEP

MPI Rank 0

solve local cells
 $u \cdot p \cdot \text{turbulence}$

MPI Rank 1

solve local cells
 $u \cdot p \cdot \text{turbulence}$

MPI Rank 2

solve local cells
 $u \cdot p \cdot \text{turbulence}$

MPI Rank 3

solve local cells
 $u \cdot p \cdot \text{turbulence}$

MPI interface exchange

Only boundary/neighbor data must cross ranks

WHY IT SCALES

Most work is local

Many cells can be solved simultaneously

MPI synchronizes shared interfaces

Faster CFD solution

KEY IDEA: Parallel CFD = local cell computation + MPI exchange at subdomain interfaces

MESH-BASED SIMULATION OF A CAR (CFD EXAMPLE)

Mesh-based (continuum) simulation solves the governing equations on a discretized domain (mesh) to predict flow behavior around the car.

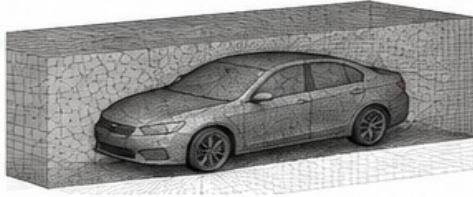
1. GEOMETRY

3D CAD model of the car



2. MESH (DISCRETIZATION)

Computational domain divided into millions of small cells



3. PHYSICS & BOUNDARY CONDITIONS

- Governing Equations: Navier-Stokes
- Turbulence Model: $k-\omega$ SST
- Inlet: Velocity = 30 m/s
- Outlet: Pressure = 0 Pa (gauge)
- Ground: No-slip wall
- Car Surface: No-slip wall
- Air: Density = 1.225 kg/m^3 , Viscosity = $1.789\text{e-}5 \text{ Pa}\cdot\text{s}$



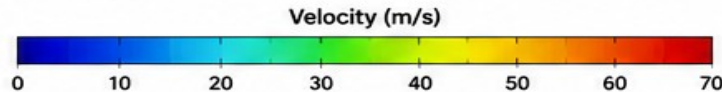
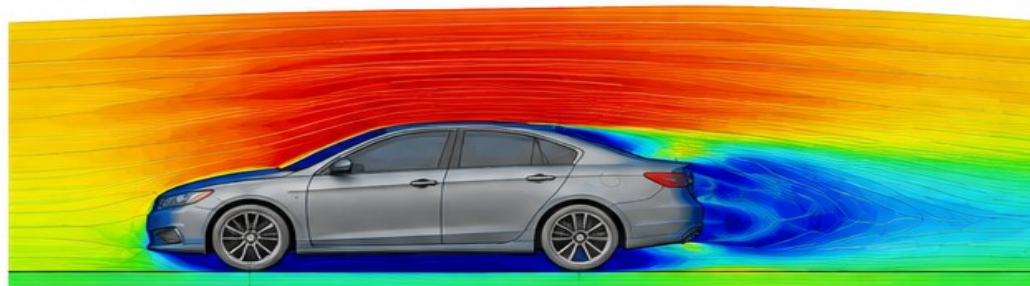
4. SOLVER (COMPUTATION)

Equations are solved iteratively on the mesh using CFD solver

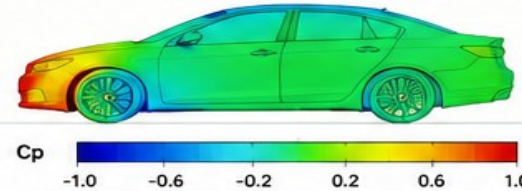
OpenFOAM®

5. RESULTS (FLOW AROUND THE CAR)

VELOCITY CONTOUR (m/s)



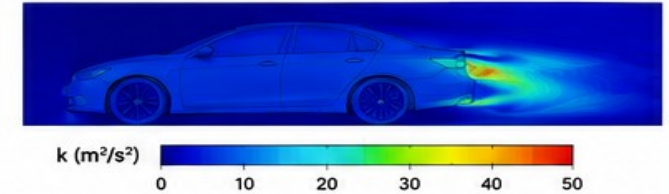
PRESSURE COEFFICIENT (C_p)



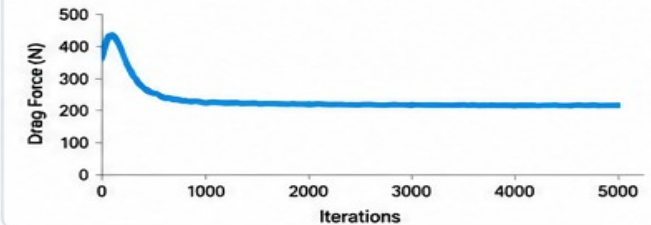
STREAMLINES



TURBULENT KINETIC ENERGY (m^2/s^2)



DRAG FORCE vs ITERATIONS



KEY OUTPUTS



Drag Force
198.7 N



Lift Force
35.2 N



Drag Coefficient (C_d)
0.294



Lift Coefficient (C_l)
0.052

APPLICATIONS

- ✓ Improve fuel efficiency (reduce drag)
- ✓ Enhance vehicle stability and handling
- ✓ Optimize shape for better aerodynamic performance
- ✓ Support design decisions before physical testing



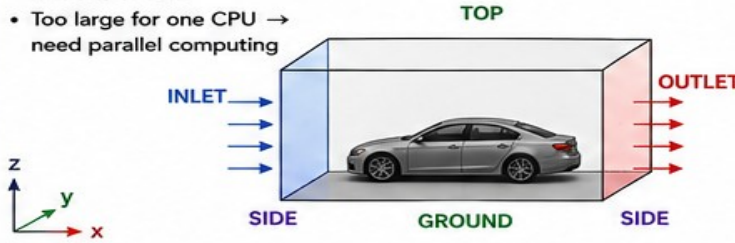
Mesh-based CFD simulation helps engineers predict and optimize airflow around a car with high accuracy before building prototypes.

DOMAIN DECOMPOSITION IN PARALLEL CFD (CAR EXAMPLE)

Divide the computational domain (air box) into smaller subdomains so different processors can solve them simultaneously.

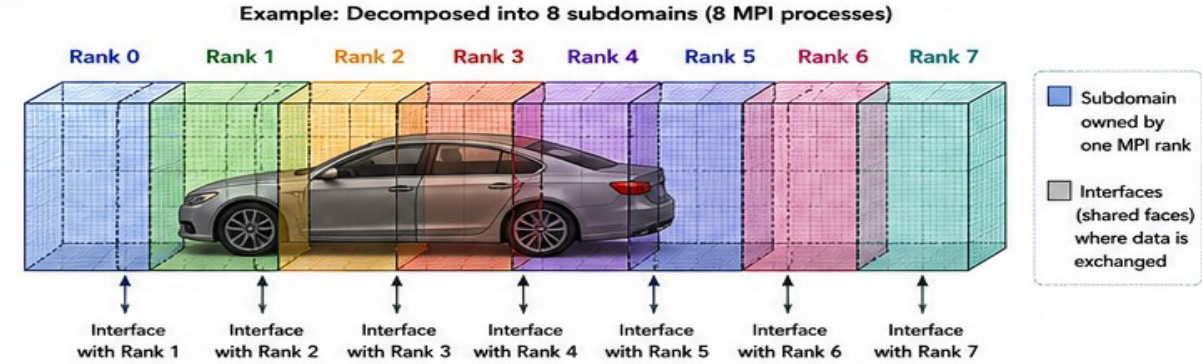
1. FULL COMPUTATIONAL DOMAIN

- Air flows from inlet to outlet
- Navier–Stokes equations are solved in the entire air box
- Millions of cells
- Too large for one CPU → need parallel computing



2. IDEA OF DOMAIN DECOMPOSITION

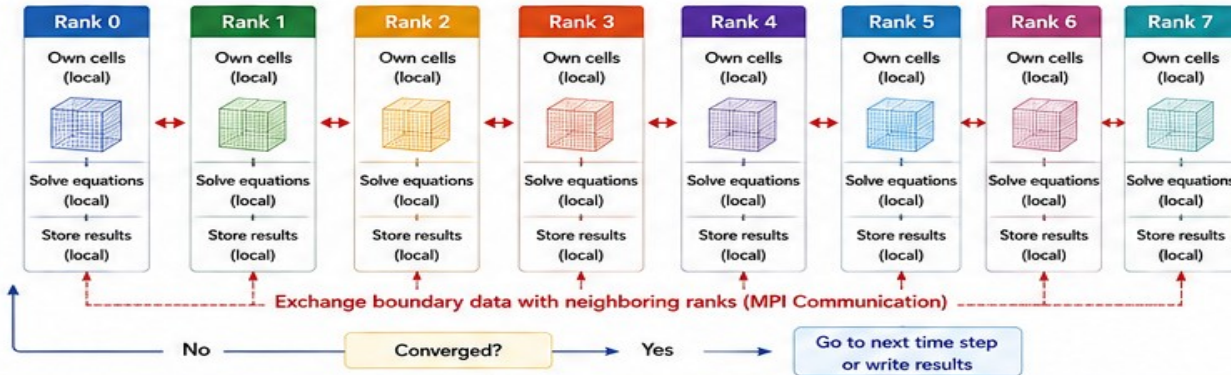
- Split the air box into N non-overlapping subdomains
- Each subdomain is assigned to one processor (MPI rank)
- Each processor solves the flow equations only in its own subdomain
- Processors exchange data on their common interfaces (boundaries)



3. HOW EACH PROCESSOR WORKS (At Each Iteration / Time Step)

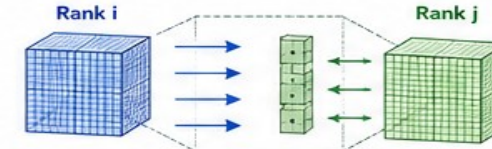
Every MPI rank performs:

- 1 Solve momentum equations in its own cells
- 2 Solve pressure equation
- 3 Update velocity
- 4 Solve turbulence equations
- 5 Compute fluxes
- 6 Check residuals



4. INTERFACE / DATA EXCHANGE

Neighboring subdomains share data on their common faces.



Exchange variables:

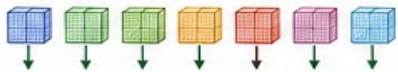
- Velocity (u, v, w)
- Pressure (p)
- Turbulence variables ($k, \omega, \dots$)
- Fluxes, gradients, etc.

Why communication is needed?

- Flow is continuous across interfaces
- Information from neighbors is required to compute correct solution
- More processors → more communication

5. EFFECT OF GOOD DECOMPOSITION

Good Decomposition (Balanced)



- Almost equal number of cells per rank
- All processors finish at similar time
- High efficiency and speedup

Bad Decomposition (Imbalanced)



- Some ranks have many more cells
- They take longer → others wait
- Low efficiency

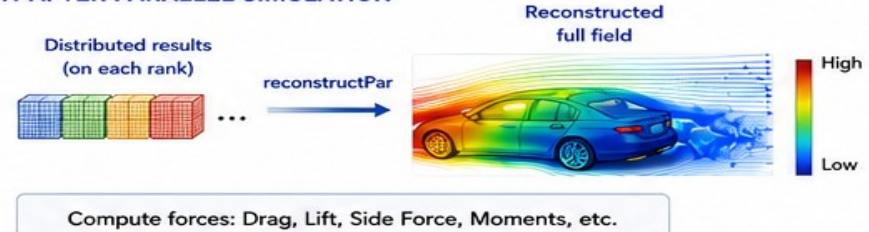
6. EXAMPLE NUMBERS (Illustration)

- Total cells in domain : 20 million
- Number of MPI ranks : 8
- Cells per rank (ideal) : 2.5 million
- Iterations : 5,000
- Variables per cell : 6 (u, v, w, p, k, ω)

Benefits

- Less memory per rank
- Less computation per rank
- Solve faster
- Ability to handle much larger problems

7. AFTER PARALLEL SIMULATION



KEY TAKEAWAY:

Domain decomposition splits the large computational domain into smaller subdomains. Each processor solves the Navier–Stokes equations in its own subdomain and communicates with neighbors. This enables efficient use of HPC resources and reduces the time to solution from days/weeks to hours.

OpenFOAM tools

Decompose: `decomposePar`
Run: `mpirun -np N simpleFoam -parallel` (or `srun -n N ...`)
Reconstruct: `reconstructPar`

SECTION RECAP

Four ideas to remember

- 1 A model is a purposeful simplification of reality.
- 2 Simulation executes that model under chosen conditions.
- 3 Four main paradigms: mesh, particle, agent/event, hybrid/multiscale.
- 4 Larger, finer and longer simulations create the need for HPC.

NEXT: What is High-Performance Computing — and how do we parallelize scientific simulation?